

Airline Management System

Introduction

In today's fast-paced world, air travel has become one of the most essential modes of transportation, connecting millions of passengers across cities, countries, and continents every day. Managing the vast amount of data involved in airline operations — from passenger bookings and flight schedules to crew assignments and baggage handling — has become a critical challenge for the aviation industry.

Our project focuses on designing and developing a Database Management System (DBMS) for an Airline Management System.

This system aims to efficiently manage:

- Passenger registration and profiles
- Flight scheduling and route management
- Ticket booking and seat allocation
- Payment processing and transaction records
- Baggage tracking and handling
- Crew assignment and role management
- Airport departure and arrival coordination

The main objective of this project is to design a well-structured, normalized relational database that ensures:

- Data accuracy
- Reduced redundancy
- Proper relationships between entities
- Efficient data retrieval
- Scalability for future expansion

By implementing meaningful entities such as Passenger, Flight, Booking, Ticket, Staff, Aircraft, and Payment, our system provides a strong foundation for a real-world airline management platform.

This project demonstrates how DBMS concepts like Entities, Attributes, Primary Keys, Foreign Keys, Relationships, and Normalization are applied in designing a practical, efficient, and scalable airline management system.

ENTITIES AND ATTRIBUTES

1 Passenger

Attributes:

- Passenger_ID (PK)
- Name
- Age
- Gender
- Phone
- Email
- Passport_No

2 Booking

Attributes:

- Booking_ID (PK)
- Booking_Date
- Seat_Number

3 Payment

Attributes:

- Payment_ID(PK)
- Amount
- Payment_Date
- Payment_Method

4 Ticket

Attributes:

- Ticket_ID(PK)
- Price
- Ticket_Type

- Status

5 Flight

Attributes:

- Flight_ID (PK)
- Flight_Name
- Departure_Time
- Arrival_Time
- Available_Seats

6 Baggage

Attributes:

- Baggage_ID (PK)
- Weight
- Type

7 Crew

Attributes:

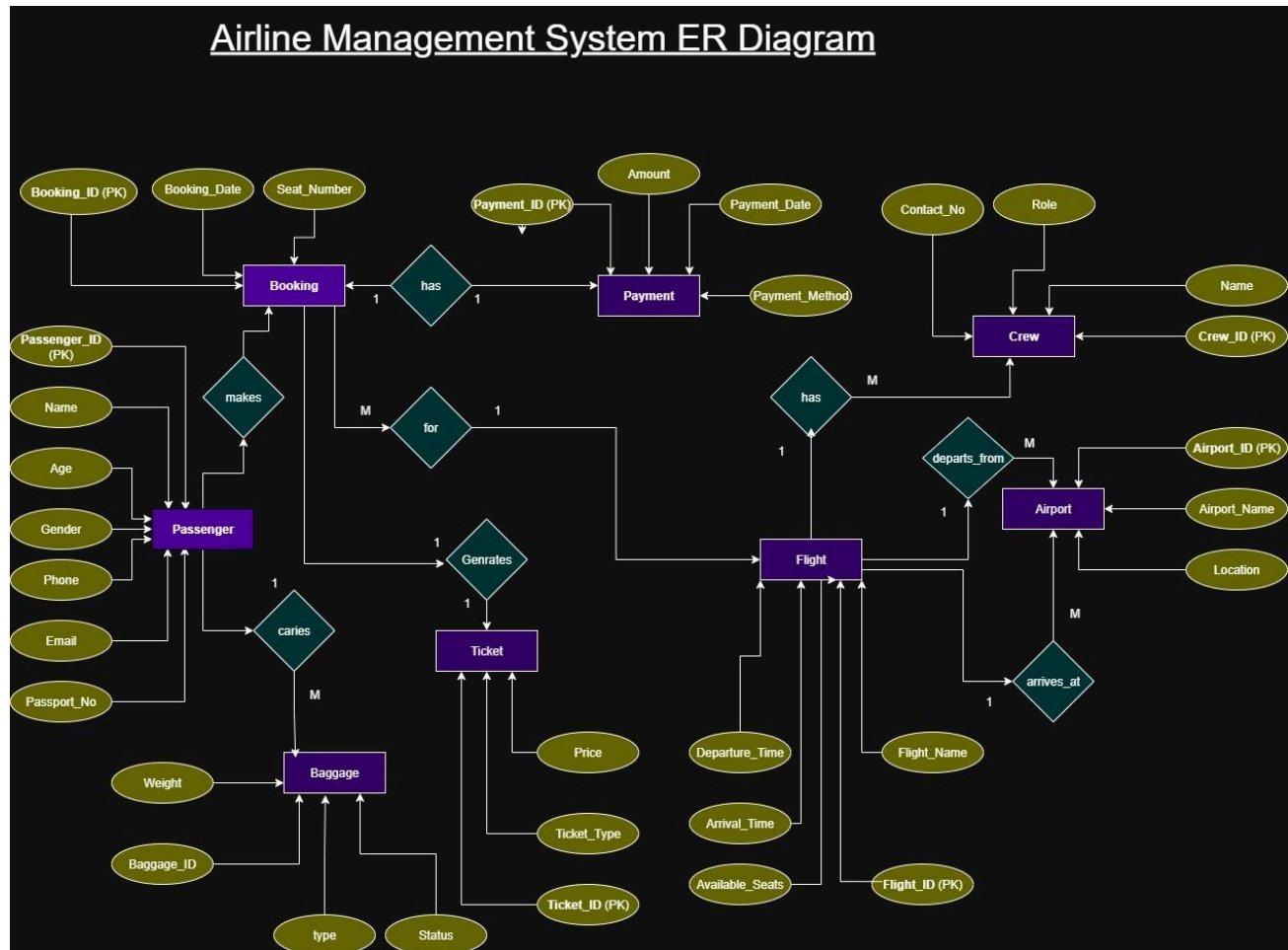
- Crew_ID (PK)
- Name
- Role
- Contact_No

8 Airport

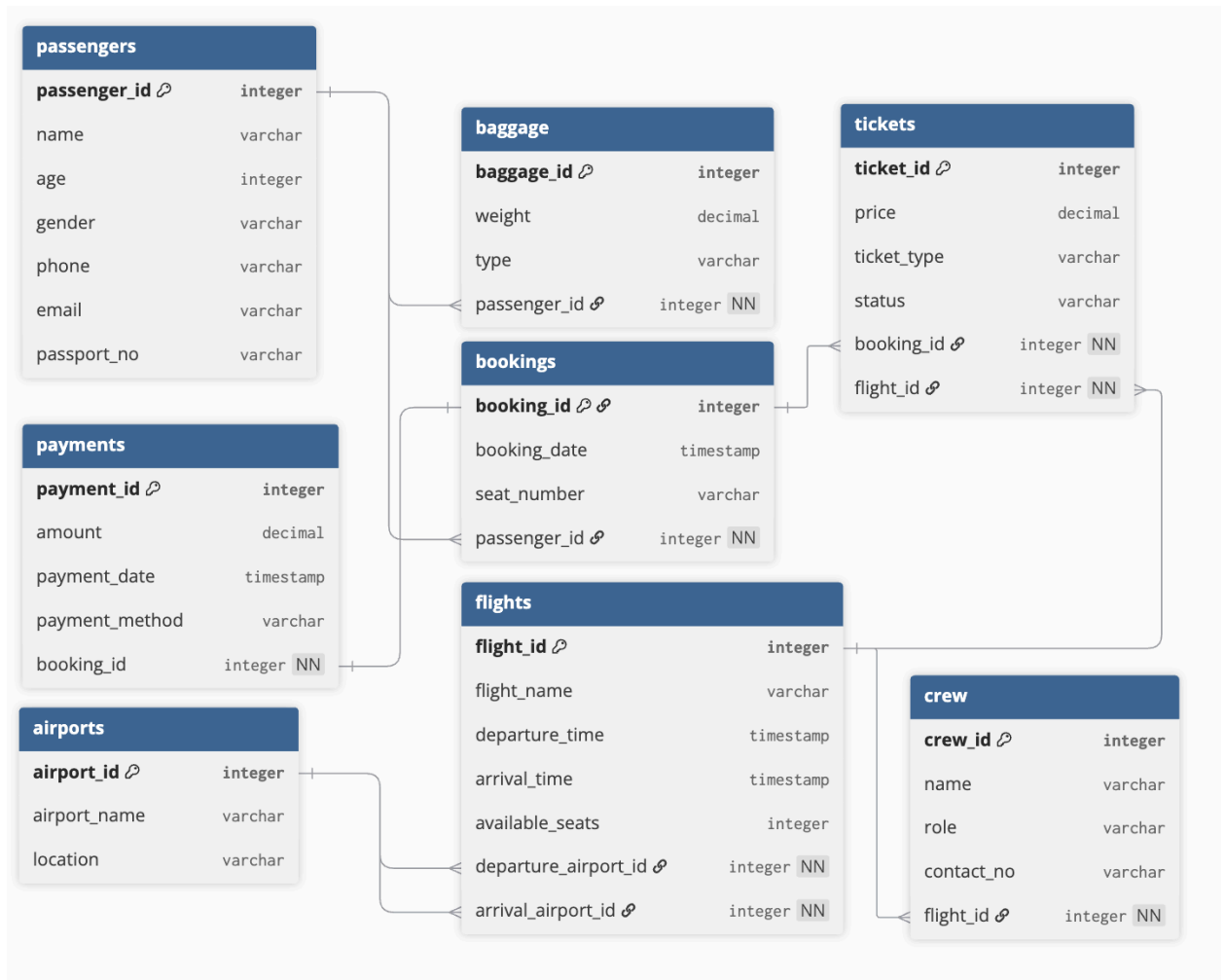
Attributes:

- Airport_ID (PK)
- Airport_Name
- Location

ER DIAGRAM :



TABLES:



1. Introduction to Normalization

Normalization is the process of organizing a relational database to reduce data redundancy and improve data integrity. For an Airline Management System that handles passengers, flights, bookings, payments, crew, and airports, normalization is essential to keep the data clean, consistent, and easy to maintain.

The main objectives of normalization are:

- Eliminate redundant (duplicate) data
- Ensure data dependencies are logical and consistent
- Improve data integrity across all tables
- Make the database scalable and easy to maintain
- Speed up queries by reducing unnecessary data repetition

2. Original Unnormalized Schema (UNF)

Before normalization, all airline data was stored in a single flat table. This caused massive repetition and inconsistency problems.

Booking_ID	Passenger	Passport	Flight	Dep_Time	Airport	Seat	Ticket_Type	Amount	Payment	Baggage	Crew	Role
1	Alice	P123	AI-101	2026-05-01 06:00	Delhi IGI	12A	Economy	5500	Credit Card	23 kg	Raj Mehta	Pilot
2	Bob	P456	AI-101	2026-05-01 06:00	Delhi IGI	14B	Economy	4800	UPI	7 kg	Raj Mehta	Pilot
3	Carol	P789	AI-202	2026-05-02 10:00	Mumbai CSI	9C	Business	6200	Net Banking	20 kg	Anil Verma	Pilot

✗ Problems: Flight info (AI-101, Delhi IGI, 06:00) repeats for every booking. Crew name repeats. Multiple baggage items would need multiple columns. No primary key uniqueness.

3. Normalized Schema Summary

After applying 1NF, 2NF, and 3NF, the database is broken into 8 clean tables:

Table	Attributes	Primary Key	Foreign Keys
Passenger	passenger_id, name, age, gender, phone, email, passport_no	passenger_id	—
Booking	booking_id, booking_date, seat_number, passenger_id	booking_id	passenger_id
Payment	payment_id, amount, payment_date, payment_method, booking_id	payment_id	booking_id
Ticket	ticket_id, price, ticket_type, status, booking_id, flight_id	ticket_id	booking_id, flight_id
Flight	flight_id, flight_name, departure_time, arrival_time, available_seats, departure_airport_id, arrival_airport_id	flight_id	departure_airport_id, arrival_airport_id
Baggage	baggage_id, weight, type, passenger_id	baggage_id	passenger_id
Crew	crew_id, name, role, contact_no, flight_id	crew_id	flight_id
Airport	airport_id, airport_name, location	airport_id	—

4. First Normal Form (1NF)

Rule: Every column must contain atomic (single) values. No repeating groups. Every row must be unique.

■ Before 1NF — Violation

Booking_ID	Passenger_Name	Flights_Booked	Baggage_Items
1	Alice	AI-101, AI-202	23 kg, 7 kg
2	Bob	AI-303	15 kg

✗ **Flights_Booked and Baggage_Items contain multiple values in one cell — this violates 1NF.**

✓ After 1NF — Fixed

Booking_ID	Passenger_Name	Flight_Name	Baggage_Weight
1	Alice	AI-101	23 kg
2	Alice	AI-202	7 kg
3	Bob	AI-303	15 kg

✓ **Each cell now holds one value. Multi-valued columns are split into separate rows.**

How 1NF is applied in our Airline System:

- Each booking is stored as a separate row in the Booking table — not as a list inside Passenger.
- Each baggage item is a separate row in the Baggage table with its own Baggage_ID.
- Each crew member is a separate row in the Crew table.
- All tables have a unique primary key (passenger_id, booking_id, flight_id, etc.).

5. Second Normal Form (2NF)

Rule: Must be in 1NF + every non-key attribute must depend on the **whole** primary key. No partial dependencies allowed.

■ Before 2NF — Violation (Composite Key: Booking_ID + Flight_ID)

Booking_ID	Flight_ID	Flight_Name	Departure_Time	Seat_Number
1	101	AI-101	2026-05-01 06:00	12A
2	101	AI-101	2026-05-01 06:00	14B
3	202	AI-202	2026-05-02 10:00	9C

✗ **Flight_Name** and **Departure_Time** depend only on **Flight_ID**, not on the full composite key (**Booking_ID + Flight_ID**). This is a partial dependency — violates 2NF.

✓ After 2NF — Fixed: Split into two tables

Booking Table:

Booking_ID	Booking_Date	Flight_ID	Seat_Number	Passenger_ID
1	2026-05-01	101	12A	P123
2	2026-05-01	101	14B	P456
3	2026-05-02	202	9C	P789

Flight Table:

Flight_ID	Flight_Name	Departure_Time	Arrival_Time	Available_Seats	Departure_Airport_ID	Arrival_Airport_ID
101	AI-101	2026-05-01 06:00	2026-05-01 09:00	150	1	2
202	AI-202	2026-05-02 10:00	2026-05-02 14:00	200	2	3

✓ **Flight_Name** and **Departure_Time** now live in the **Flight** table where they fully depend on **Flight_ID**.

How 2NF is applied in our Airline System:

- Booking table only stores booking-specific data (booking_date, seat_number) that

depends on booking_id.

- Flight table stores all flight details that depend fully on flight_id.
- Passenger table stores all passenger info that depends fully on passenger_id.
- Ticket table attributes (price, ticket_type, status) all depend on ticket_id alone.

6. Third Normal Form (3NF)

Rule: Must be in 2NF + no transitive dependencies. A non-key column must not depend on another non-key column.

■ Before 3NF — Violation

Flight_ID	Flight_Name	Airport_ID	Airport_Name	Location
101	AI-101	1	IGI Airport	New Delhi
202	AI-202	2	CSI Airport	Mumbai
303	AI-303	1	IGI Airport	New Delhi

✗ Airport_Name and Location depend on Airport_ID (a non-key column), not on Flight_ID. This is a transitive dependency — violates 3NF. IGI Airport / New Delhi repeats for every flight.

✓ After 3NF — Fixed: Split into two tables

Flight Table:

Flight_ID	Flight_Name	Departure_Time	Arrival_Time	Available_Seats	Departure_Airport_ID	Arrival_Airport_ID
101	AI-101	2026-05-01 06:00	2026-05-01 09:00	150	1	2
202	AI-202	2026-05-02 10:00	2026-05-02 14:00	200	2	3
303	AI-303	2026-05-03 12:00	2026-05-03 16:00	180	1	3

Airport Table:

Airport_ID	Airport_Name	Location
1	IGI Airport	New Delhi
2	CSI Airport	Mumbai
3	Kempegowda International	Bengaluru

✓ **Airport_Name and Location are now in the Airport table where they depend only on Airport_ID (the PK).**

How 3NF is applied in our Airline System:

- Airport details (Airport_Name, Location) are in a separate Airport table — not inside Flight.
- Crew details (Name, Role, Contact_No) are in a separate Crew table — not inside Flight.
- Payment details are in a separate Payment table — not inside Booking.
- Each table's non-key columns depend ONLY on that table's primary key.

7. Summary

Normal Form	Rule Applied	Example in Airline System
1NF	Atomic values, no repeating groups	Each booking = one row; each baggage item = one row

Normal Form	Rule Applied	Example in Airline System
2NF	No partial dependencies on composite key	Flight details moved to Flight table; not repeated in Booking
3NF	No transitive dependencies	Airport info moved to Airport table; Crew info to Crew table

By applying all three normal forms, the Airline Management System database is clean, consistent, and free of redundancy. Any update (e.g., changing an airport name or a flight's departure time) only needs to happen in one place, keeping all related data automatically consistent.